

Building GPT Models Through Classical Arabic Poetry

An Undergraduate Workshop Series on Generative Pre-trained Transformers

Eng. Mohammad Reda Katby



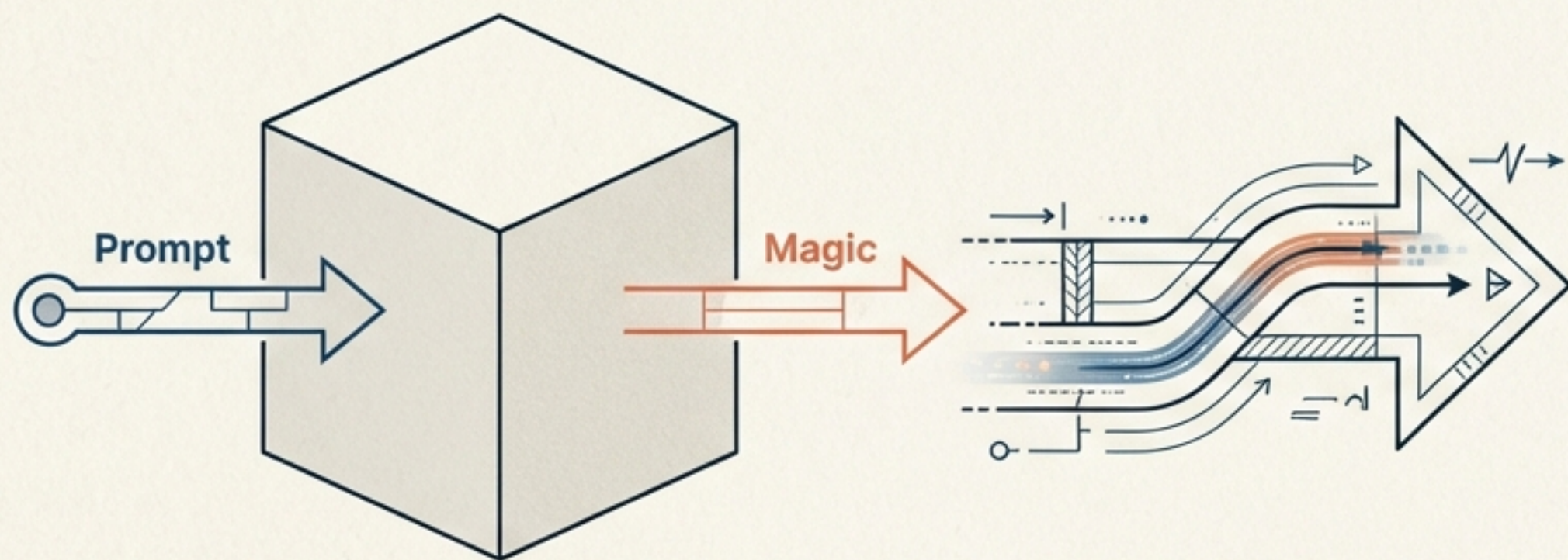
PyTorch tensor

...	...
24 85 102	24 85 102
13 56 99	13 56 99
72 41 10	72 41 10
11 33 66	11 33 66
23 86 97	23 83 97
13 82 34	13 82 34
.. . ..	
72 41 10	72 41 10
11 33 66	11 33 66
11 33 66	24 85 102

PyTorch tens.

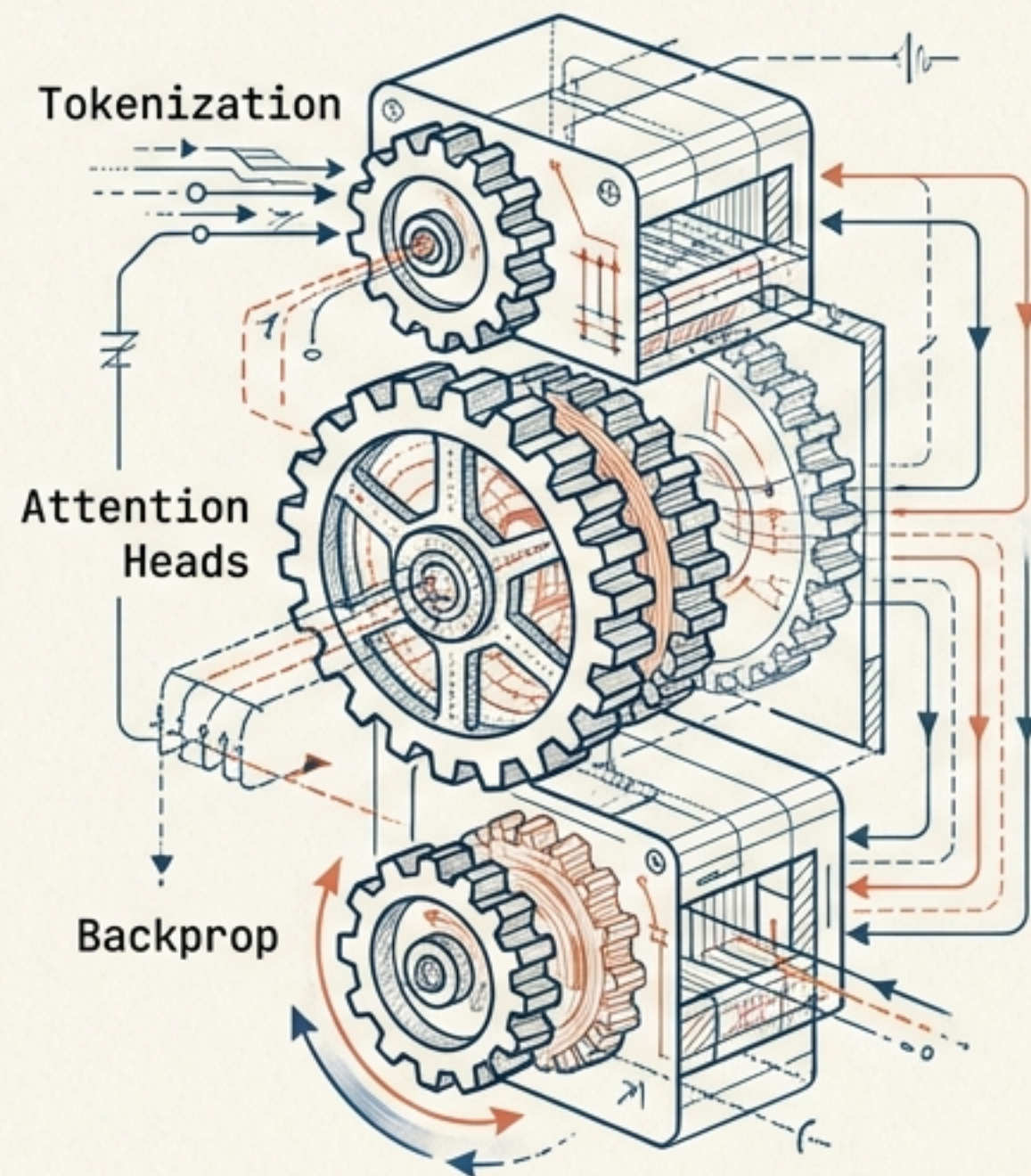
Moving from Consumers of Magic to Architects of Math

The Consumer View

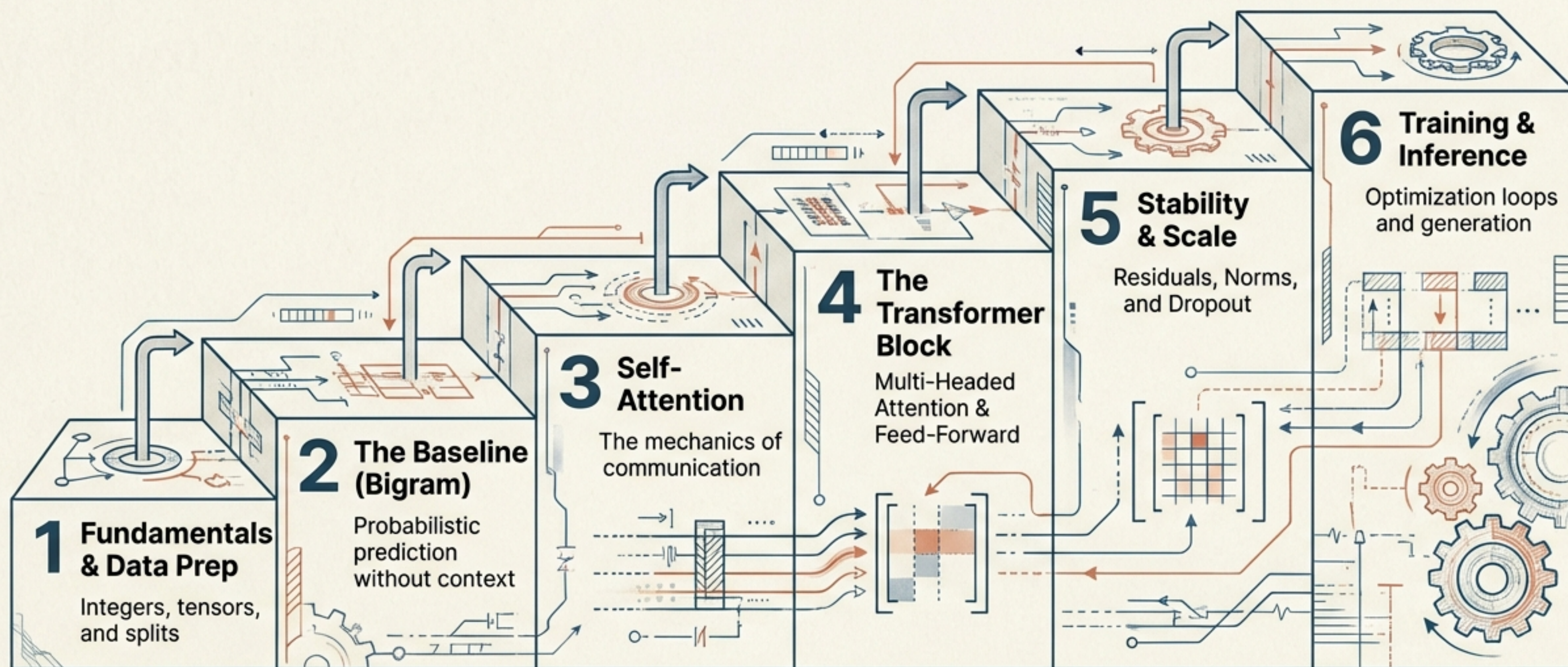


The workshop objective is to demystify Large Language Models (LLMs) by implementing a **decoder-only Transformer** from scratch. We are teaching the architecture of GPT-3 using poetry as the data substrate.

The Architect View



Six Modules to Build a Generative Mind



Each module combines theoretical lectures with live coding in Python/PyTorch.

Computers Don't Read Poetry, They Read Integers

The Tokenizer Strategy

Industry Standard (OpenAI):
TikToken (50,000+ tokens,
sub-word).

Our Approach: Character-level
Tokenizer (~65 tokens for English,
adapted for Arabic alphabet).

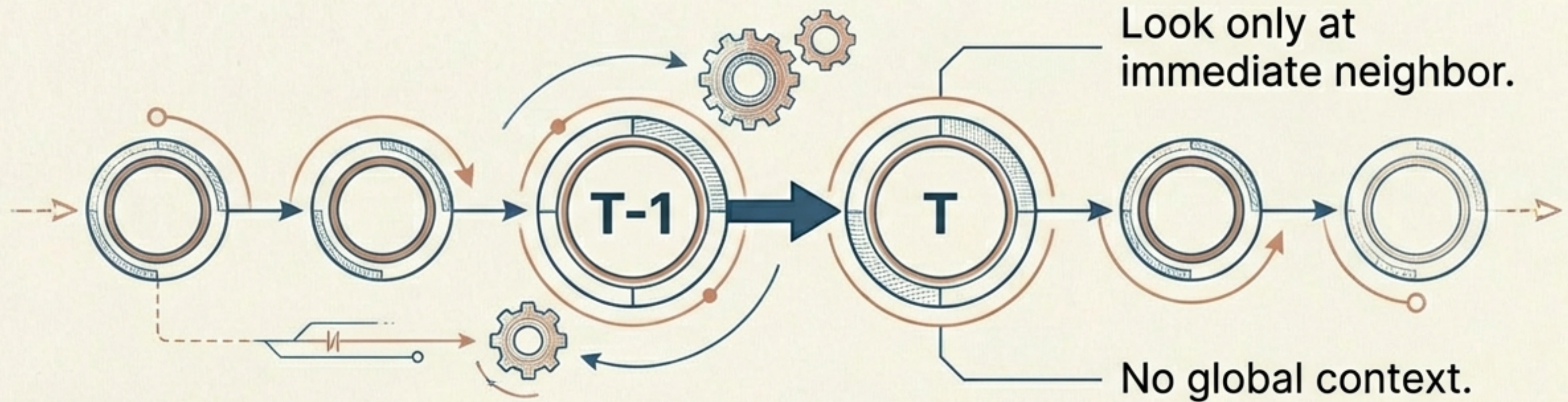
Trade-off: Simpler codebook,
longer sequence lengths.

Data Engineering

```
vocab = sorted(list(set(text)))  
stoi = { ch:i for i,ch in enumerate(vocab) }  
encode = lambda s: [stoi[c] for c in s]
```



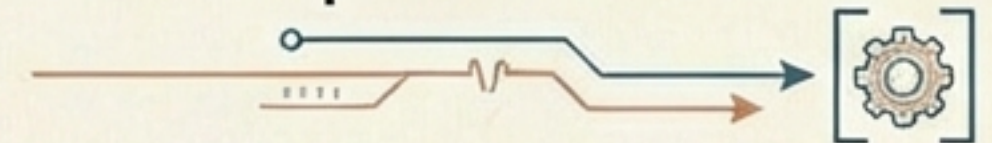
The Bigram Baseline: Prediction Without Memory



Metric: Cross-Entropy Loss (Negative Log Likelihood).



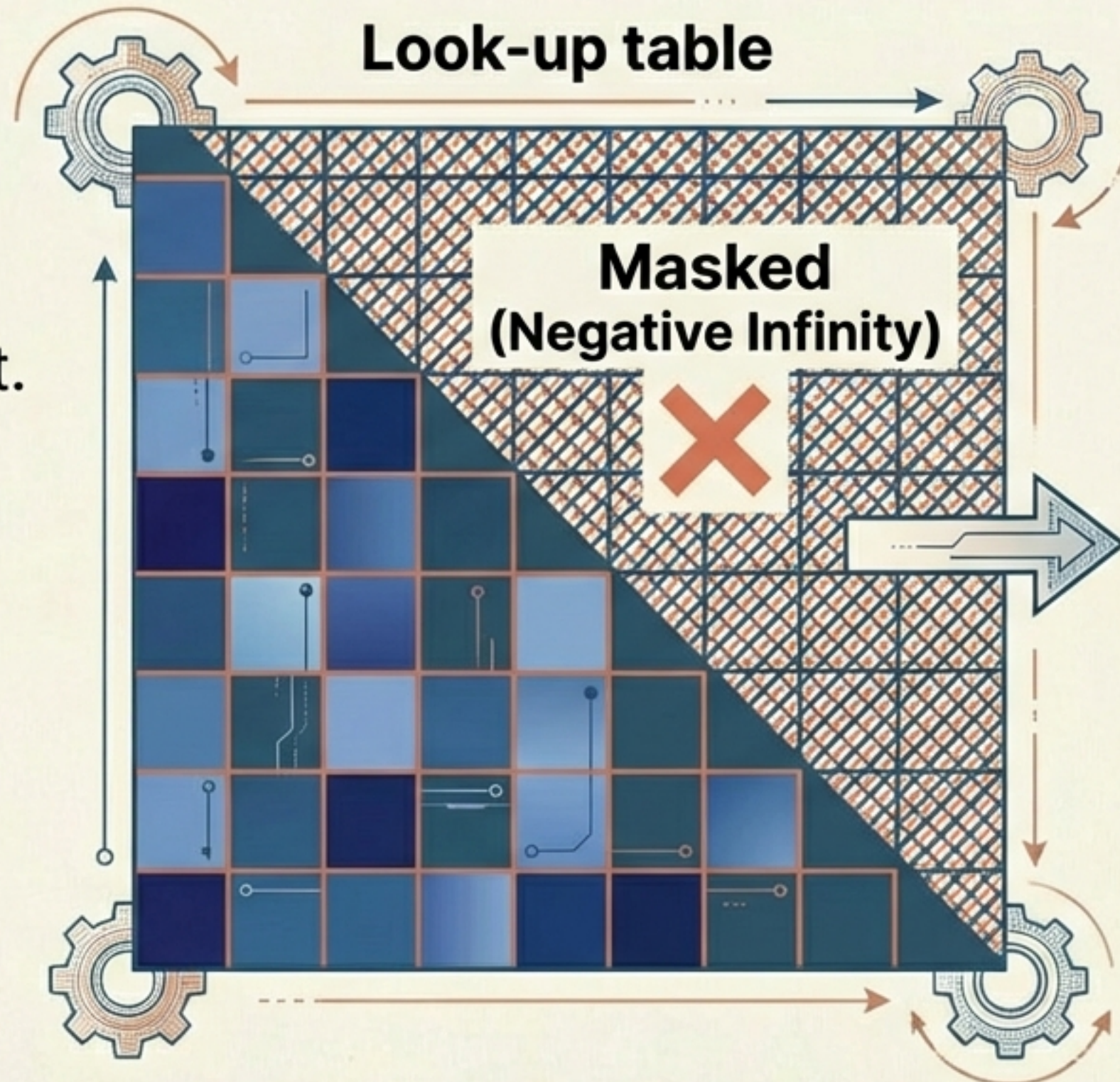
Outcome: The output is 'garbage'—random characters. This proves the need for a mechanism to look further back in time.



Solving the Communication Problem with Self-Attention

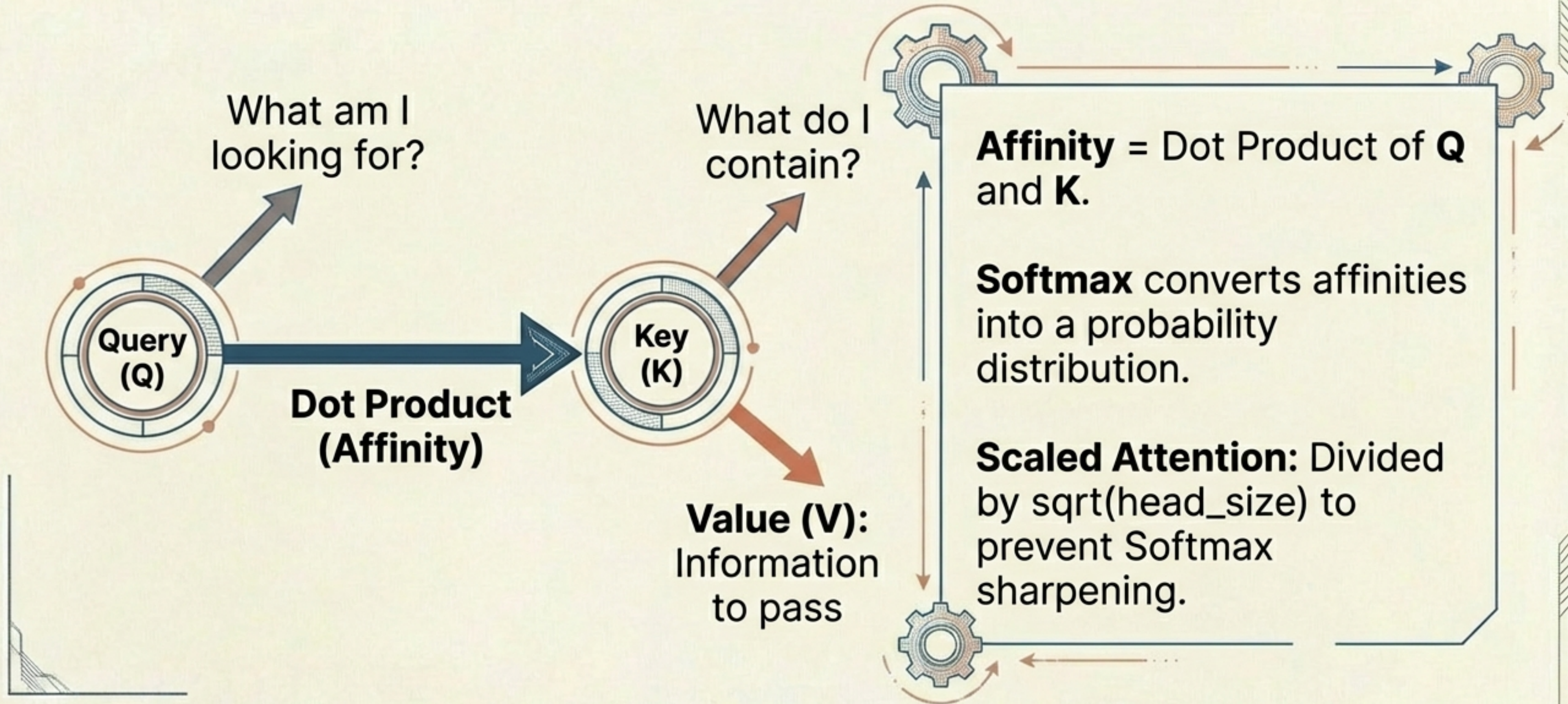
To write poetry, a token needs to 'attend' to the entire verse preceding it.

We use Matrix Multiplication to aggregate information from the past, from the past, ensuring information only flows forward in time.

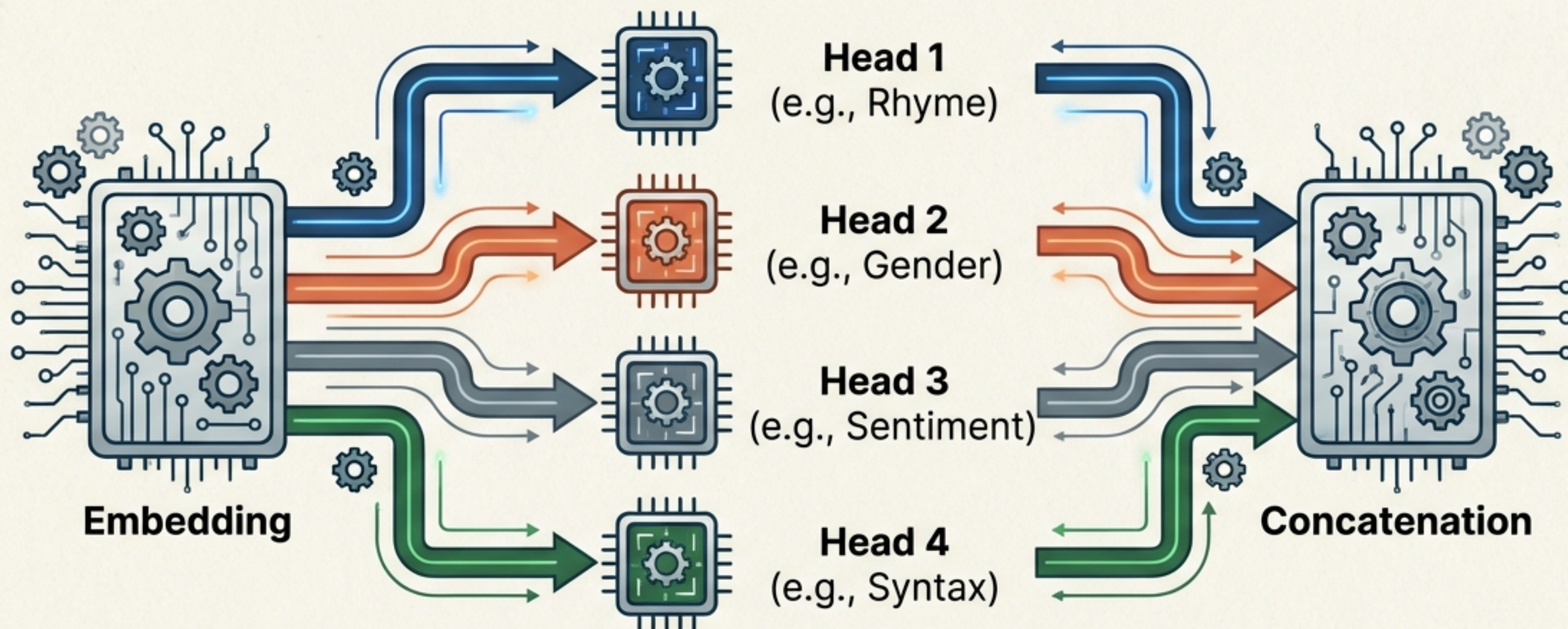


Auto-regressive property: The model cannot see the future.

Queries, Keys, and Values: A Search Retrieval System

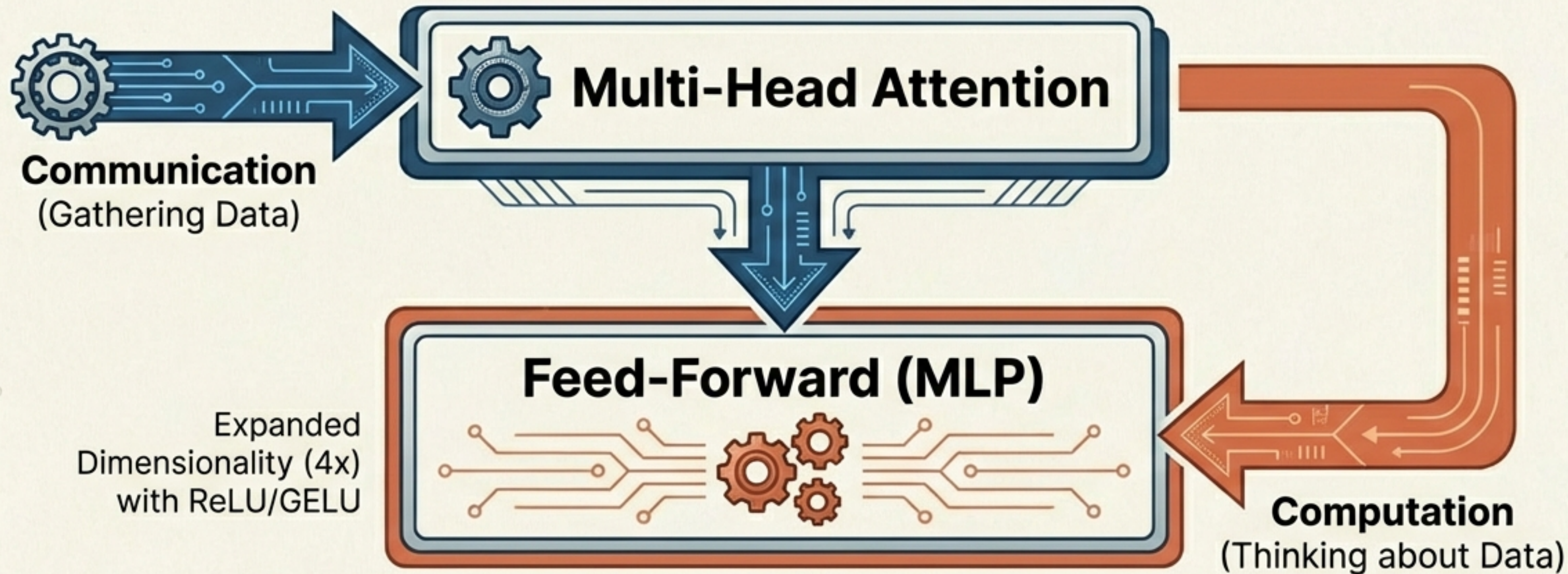


Multi-Head Attention: Parallel Conversations



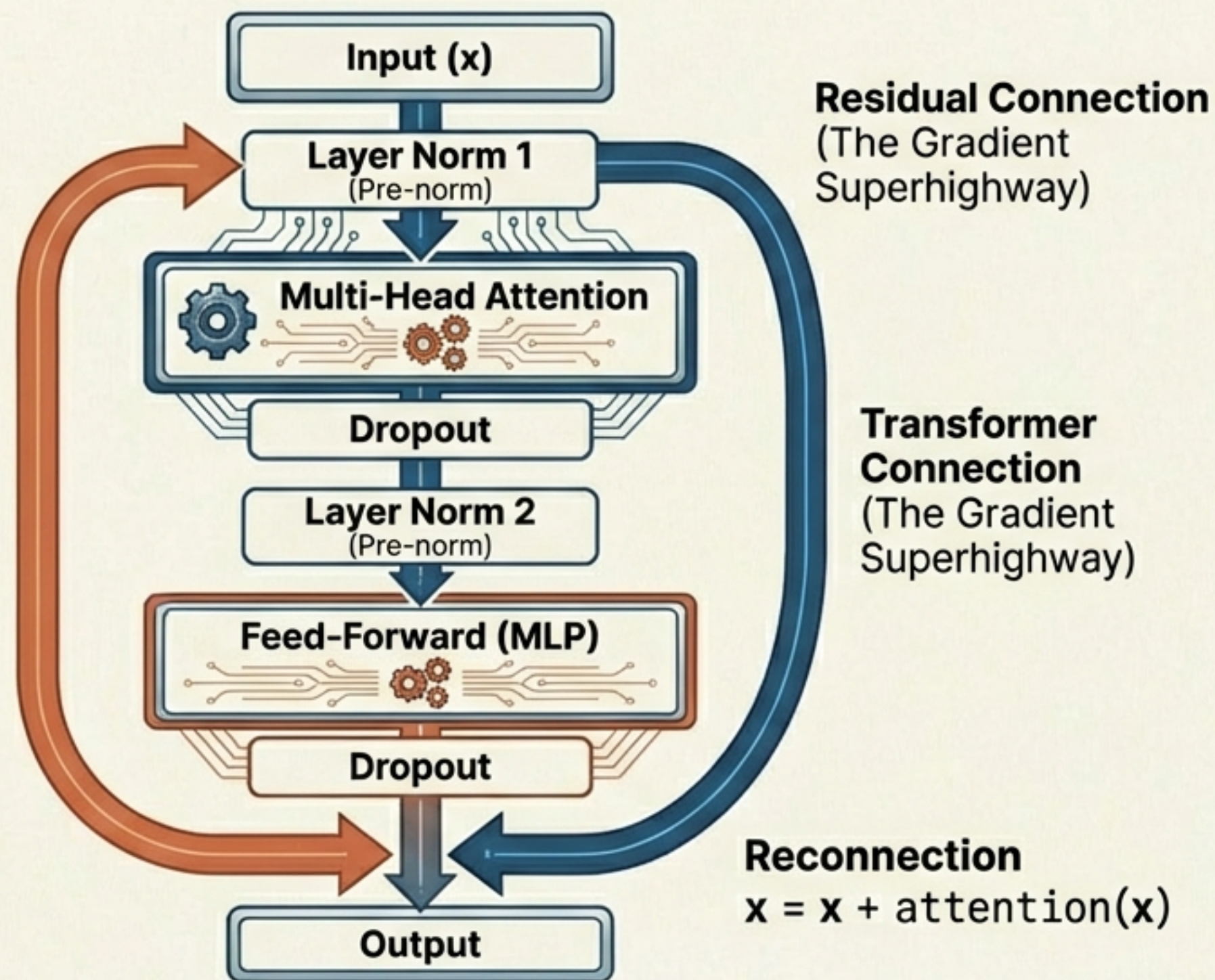
Analogy: Group Convolution. Parallel processing allows the model to learn different linguistic features simultaneously.

The Feed-Forward Network: The 'Thinking' Phase



Tokens process their gathered information independently before passing it to the next layer.

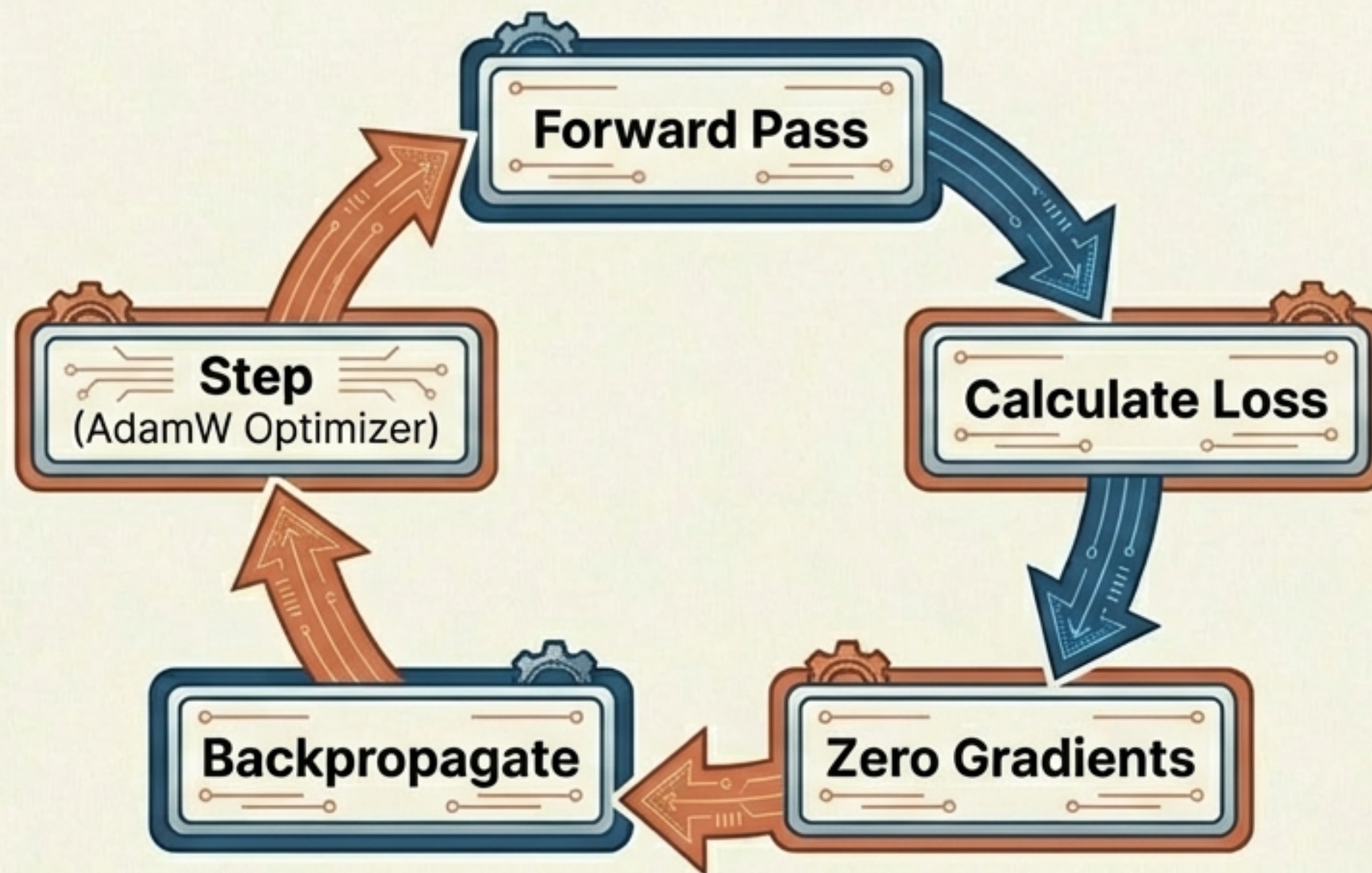
Engineering Stability with Residuals and Norms



Side Notes

- **Residual Connections:** Allow gradients to flow unimpeded, solving the vanishing gradient problem.
- **Layer Normalization (Pre-norm):** Stabilizes training dynamics.
- **Dropout:** Randomly shuts off neurons to prevent rote memorization.

The Optimization Loop and Generation



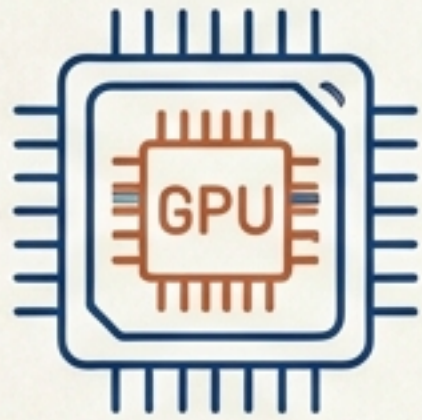
Inference Strategy: The model outputs logits, we apply Softmax for probabilities, and sample (`torch.multinomial`) to generate infinite verses.

NanoGPT vs. GPT-3: Identical DNA, Different Scale

Feature	Our Model (NanoGPT)	GPT-3 (OpenAI)
Parameters	~10 Million	175 Billion
Training Tokens	~1 Million (Characters)	300 Billion (Tokens)
Architecture	Decoder-only Transformer	Decoder-only Transformer

Key Takeaway: We are building a 'baby' version of the real thing. The logic, code structure, and mathematical foundations are identical to state-of-the-art systems.

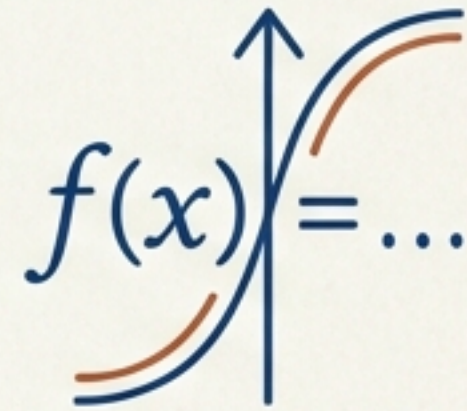
Technical Requirements & Prerequisites



Hardware: GPU
Recommended (Google
Colab / A100) for
training 8+ layers.



Software Stack:
Python 3 + PyTorch.



Prerequisites: Python
proficiency + Intro
Calculus & Statistics
(Chain rule, derivatives).

Learning Outcomes: From Theory to Tensor

Hard Skills (Engineering)

Inter

- Mastery of PyTorch tensor manipulation (shapes, views, broadcasting).
- Implementation of Self-Attention and Transformer blocks.
- Understanding Language Modeling.

Soft Skills (Synthesis)

Inter

- Bridging classical humanities with modern AI engineering.
- Conceptualizing AI not as magic, but as a sequence of matrix multiplications.

Final Product: A functional generative model producing stylistically consistent Arabic verse.